

Tekrarlı İşlemler ve Kontrol Akışı

Görsel Programlama - Hafta 11

Giriş: Tekrarlı İşlemler ve Kontrol Akışı

Bilgisayar programlarının temel taşlarından biri kontrol akışıdır (Control Flow). Kontrol akışı, kodun hangi sırada çalışacağını belirler.

İki temel kontrol akışı türü vardır: Seçim (if/else) ve Tekrarlama/Döngü (Loops).

Döngüler, bir kod bloğunu belirli bir koşul sağlanana kadar defalarca çalıştırmayı sağlar.

Tekrarlı işlemlerin otomasyonu, yazılım verimliliği ve performansı açısından kritiktir.



Döngülere Neden İhtiyaç Duyarız?

Verimlilik: Yüzlerce veya binlerce satırlık tekrarlı kodu tek bir döngü yapısıyla özetlemek.

Dinamiklik: Çalışma anında belirlenen veri setleri (listeler, diziler) üzerinde işlem yapabilme.

Algoritmik Temel: Sıralama, arama, veri işleme gibi temel algoritmaların tamamı döngülere dayanır.

Kaynaklar, döngüleri belirli bir başlangıç, koşul ve artış/azalış miktarına göre tekrarlı işlemler yapmak için kullanılan yapılar olarak tanımlar.

C# Döngü Çeşitleri ve 'for' Döngüsünün Konumu

C#'ta temel olarak üç ana döngü türü kullanılır:

1. for Döngüsü: İterasyon sayısı önceden bilindiğinde (veya başlangıç, bitiş ve adım net olduğunda).
 2. while Döngüsü: Koşul doğru olduğu sürece çalışır, iterasyon sayısı belirsiz olabilir.
 3. foreach Döngüsü: Koleksiyonlardaki her bir öge üzerinde gezinmek için kullanılır.
- 'for' döngüsü, özellikle matematiksel diziler ve belirli sınırlar içindeki tekrarlar için tercih edilir.

C# 'for' Döngüsü: Temel Syntax

for döngüsü, üç temel ifadeyi tek bir satırda toplar:

'for (Başlatma; Koşul; İterasyon)'

Başlatma (Initialization): Döngü değişkeninin (genellikle 'i') başlangıç değeri belirlenir. Yalnızca bir kez çalışır.

Koşul (Condition): Döngünün her iterasyonundan önce kontrol edilen boolean ifade. Koşul doğru olduğu sürece döngü çalışır.

İterasyon (Iteration/Step): Her döngü sonu çalıştırılan ifade (artırma/azaltma).



C# 'for' Anahtar Kelimesi Detaylı İnceleme

Kaynakta belirtilen standart C# yapısı:

```
'for (int i = 0; i < limit; i++) { ... }'
```

'int i = 0': İterasyon değişkeni 'i' sıfırdan başlar.

'i < limit': Döngü, 'i' limitten küçük olduğu sürece devam eder.

'i++': Her adımda 'i' bir artırılır (Postfix increment).

Bu yapı, sıfır tabanlı indeksleme kullanan koleksiyonlarda yaygın olarak kullanılır.



Döngü Değişkeninin Kapsamı (Scope)

Genellikle 'for' döngüsü içinde tanımlanan değişken ('int i'), döngü bloğunun dışında erişilemez (Block Scope).

'for (int i = 0; ...)' yapısı, değişkenin yalnızca o döngüye ait olmasını sağlar. Bu, kodun temizliğini ve olası değişken çakışmalarını (shadowing) engeller. Modern C# uygulamalarında, genellikle 'var' yerine 'int' gibi açık tip tanımlaması tercih edilir.



Döngü Akışı Görselleştirme

1. Başlatma (Initialization) çalışır (Sadece 1 kez).
2. Koşul kontrol edilir.
3. Koşul DOĞRU ise, döngü gövdesi çalışır.
4. İterasyon ifadesi (artırma/azaltma) çalışır.
5. 2. adıma geri dönülür.
6. Koşul YANLIŞ olduğunda, döngü sona erer ve akış döngüden sonraki koda geçer.

Örnek 1: Geriye Sayım

for döngüsü sadece artan sayılar için kullanılmaz.

Geriye sayım veya azalan diziler oluşturmak için de idealdir.

Örnek: 10'dan 1'e geri sayım

'for (int i = 10; i >= 1; i--)'

Bu durumda, Koşul 'i >= 1' olarak, iterasyon ise 'i--' (azaltma) olarak ayarlanır.

Örnek 2: Çift Sayıları Listeleme

Standart 'i++' (bir artırma) yerine, iterasyon adımını değiştirebiliriz.

Örnek: 0'dan 50'ye kadar çift sayılar.

'for (int i = 0; i <= 50; i = i + 2)' veya 'for (int i = 0; i <= 50; i += 2)'

Bu, her adımda döngü değişkeninin iki artırıldığı anlamına gelir.

Alternatif olarak, 'if' kontrolü kullanarak da çift/tek sayılar ayrılabilir, ancak bu yöntem daha performanslıdır.

Sonsuz Döngüler (Infinite Loops)

Sonsuz döngü, koşulun daima doğru kalması nedeniyle döngünün hiçbir zaman sonlanmaması durumudur.

Sebep: Yanlış Koşul (örneğin: ' $i \leq 100$ ' iken ' $i = 1$ '), veya iterasyonun unutulması.

Örnek Hata: `'for (int i = 1; i <= 10; )'`

Üniversite düzeyinde, sonsuz döngülerin CPU kaynaklarını hızla tüketmesi ve programın kilitlenmesi konularına dikkat edilmelidir.

Sonsuz döngü, yalnızca belirli sunucu süreçleri gibi özel durumlarda kontrollü olarak kullanılır.

Kurumsal Uygulamalarda 'for' Döngüsü Kullanımı

Büyük veri setleri üzerinde çalışırken 'for' döngüsü, özellikle diziler (arrays) ve 'List<T>' gibi koleksiyonlarda elemanlara indeks üzerinden erişim sağladığı için yüksek performans sunar.

Performans İpuçları: Döngü koşulu içindeki karmaşık hesaplamalardan kaçının. Limit değerini döngüden önce bir değişkene atamak (caching loop limit) mikro-optimizasyon sağlar.

Örnek: `int limit = myList.Count; for (int i = 0; i < limit; i++)`

Yeni Form Kontrolü: ListBox Tanıtımı

ListBox, kullanıcı arayüzünde (UI) bir öge listesi göstermek ve kullanıcının bu listeden bir veya daha fazla öge seçmesini sağlamak için kullanılan bir kontroldür.

Tek sütunlu, kaydırılabilir bir liste görünümü sunar.

Kaynakta belirtildiği gibi, 'for' döngüleri ile oluşturulan dinamik veriyi son kullanıcıya sunmanın en basit yollarından biridir.



ListBox'ın Temel Metotları ve Özellikleri

Bir ListBox kontrolünün en önemli özelliği, içerdiği öğeler koleksiyonudur ('Items').

Bu koleksiyonu yönetmek için kullanılan ana metotlar şunlardır:

Items.Add(object item): Listeye yeni bir öğe ekler.

Items.Clear(): Listedeki tüm öğeleri anında temizler.

Items.RemoveAt(int index): Belirli bir indeksteki öğeyi siler.

Items.Count: Listedeki toplam öğe sayısını verir.

ListBox ve Veri Bağlama (Data Binding) Kavramı

Üniversite düzeyinde, ListBox'ın sadece manuel ekleme ('Items.Add') yerine, genellikle veri bağlama (Data Binding) ile kullanıldığını bilmek gerekir.

Veri Bağlama: Listenin görünümünü doğrudan bir veri kaynağına (örneğin bir 'List<string>' veya veritabanı tablosu) bağlama tekniği.

Bu yaklaşım, UI ve iş mantığını ayırarak (MVVM, MVP mimarileri) daha sürdürülebilir uygulamalar geliştirmeyi sağlar.



İlgili Form Kontrolleri

Döngülerle çalışan interaktif projelerde aşağıdaki temel kontroller kullanılır:

Button (Düğme): Kullanıcının bir işlemi (örneğin döngüyü başlatmayı) tetiklemesi için.

Label (Etiket): Kullanıcıya sonuçları veya açıklamaları göstermek için.

TextBox (Metin Kutusu): Kullanıcıdan giriş (input) almak için (örneğin hangi sayıya kadar hesaplama yapılacağı).

Proje 1: 1'den 100'e Sayıları ListBox'a Ekleme

Amaç: En basit haliyle 'for' döngüsünün nasıl çalıştığını ve bir ListBox'ı dinamik olarak nasıl doldurduğunu göstermek.

Bu proje, döngünün Başlangıç, Koşul ve İterasyon adımlarını görsel olarak anlamayı hedefler.



Proje 1 Arayüz (UI) Tasarımı

Arayüz, minimum bileşenlerle tasarlanmıştır:

1 adet ListBox (listBox1): Sonuçların gösterileceği yer.

1 adet Button (btnDoldur): İşlemi başlatacak düğme. Metni 'Doldur' olabilir.

Bu minimalist tasarım, öğrencilerin odağını UI karmaşasından ziyade döngü mantığına yönlendirir.



Proje 1: Kod Akışı Adım 1 – Temizleme

Kullanıcı her 'Doldur' düğmesine bastığında, listenin önceki verilerden arındırılması gerekir.

```
'listBox1.Items.Clear();'
```

Bu işlem, veri tutarlılığını sağlar ve her çalıştırmanın bağımsız olmasını garanti eder.

Eğer bu adım atlanırsa, her basısta 1'den 100'e sayılar listenin sonuna eklenmeye devam eder.

Proje 1: Kod Akışı Adım 2 – Döngü Tanımlama

1'den 100'e kadar sayılar listeleneceği için, döngü 1'den başlamalı ve 100 dahil olana kadar devam etmelidir.

'for (int i = 1; i <= 100; i++)'

Başlatma: 'int i = 1'

Koşul: 'i <= 100' (100'ü dahil etmek için küçük eşit kullanılır)

İterasyon: 'i++' (birer birer artırma)

Proje 1: Kod Akışı Adım 3 – Listeye Ekleme

Döngünün her adımında, mevcut 'i' değeri ListBox'a eklenir.

```
'listBox1.Items.Add(i);'
```

ListBox'ın 'Items.Add()' metodu, aldığı veriyi (bu durumda bir tamsayıyı) otomatik olarak string'e dönüştürüp gösterir.

Proje 2: Faktöriyel Hesaplama Giriş

Amaç: 'for' döngüsünü sadece listeleme için değil, aynı zamanda matematiksel bir hesaplama (çarpım) işlemi için de kullanmak.

Faktöriyel ($n!$): 1'den n 'e kadar olan tüm pozitif tam sayıların çarpımıdır (Örn: $5! = 5 * 4 * 3 * 2 * 1 = 120$).



Proje 2: Faktöriyel UI ve Kullanıcı Girişi

Arayüz Bileşenleri:

TextBox: Kullanıcının faktöriyeli hesaplanacak sayıyı girdiği alan (sayı girişi).

Button: Hesaplama işlemini başlatan düğme ('Hesapla').

Label: Sonucun metin olarak gösterileceği alan.

Kod Akışı Başlangıcı: Girilen sayı TextBox'tan alınır ve tamsayıya dönüştürülür ('int sayı').

Kritik Veri Tipi Seçimi: 'long' Kullanımı

Faktöriyel değerleri çok hızlı büyür.

Örn: 13! değeri, standart 32-bit 'int' veri tipinin maksimum değerini aşar (yaklaşık 2.1 Milyar). Bu duruma Tamsayı Taşması (Overflow) denir.

Kaynak, bu nedenle sonucun saklanması için daha geniş bir aralığa sahip olan 'long' (64-bit tamsayı) veri tipini kullanmayı önerir.

Başlangıç Değeri: 'long sonuc = 1;' (Çarpma işleminde etkisiz eleman 1'dir).

Proje 2: Döngü ve Çarpım Mantiğı

Döngü, 1'den, kullanıcının girdiği sayıya ('sayi') kadar çalışır.

```
'for (int i = 1; i <= sayi; i++)'
```

Her adımda, o anki döngü değişkeni ('i'), bir önceki birikmiş sonuç ile çarpılır:

```
'sonuc = sonuc * i;' veya daha kısa gösterimiyle 'sonuc *= i;'
```

Bu yapı, programlamada 'Accumulator Pattern' (Toplayıcı/Biriktirici Desen) olarak bilinir.

Proje 2: Sonuç Gösterimi

Hesaplama tamamlandıktan sonra, sonuç Label kontrolünde gösterilir.
Label metni, hem faktöriyeli alınan sayıyı hem de sonucu içerecek şekilde biçimlendirilmelidir
Bu, kullanıcının neyin hesaplandığını net olarak görmesini sağlar.

Akademik Bağlam: Big O Notasyonu ve Faktöriyel

Üniversite düzeyinde, bir 'for' döngüsünün zaman karmaşıklığı $O(N)$ olarak ifade edilir.

Faktöriyel hesaplaması için döngü, girdi boyutu (N) ile doğru orantılı olarak N adımda tamamlanır.

Bu, oldukça hızlı ve verimli bir algoritmadır.

Ancak, Faktöriyel fonksiyonunun matematiksel büyüme hızı çok yüksektir, bu nedenle taşma riski (Overflow) $O(N)$ karmaşıklığından bağımsız olarak mevcuttur.

Proje 3: İç İçe Döngüler (Nested Loops) Giriş

İç içe döngüler, bir döngünün gövdesinde başka bir döngü tanımlandığında ortaya çıkar.

Amaç: Bir dış döngüdeki her bir adım için, iç döngünün tüm adımlarını çalıştırmak.

Temel kullanım alanı, iki boyutlu veri yapılarını (matrisler) işlemek veya her bir elemanın diğer elemanla etkileşimini hesaplamaktır.

Proje 3 (Çarpım Tablosu), iç içe döngülere mükemmel bir örnektir.

Çarpım Tablosu: UI Tasarımı

Çarpım Tablosu projesi için arayüzde 1 adet ListBox veya RichTextBox önerilir. (ListBox daha yaygındır).

1 adet Button ('Oluştur') ile işlem tetiklenir.

RichTextBox, metni daha gelişmiş formatta gösterme imkanı sunsa da, ListBox basit liste sunumu için yeterlidir.



Çarpım Tablosu: Dış Döngü (i)

Dış döngü, tablonun hangi sayının çarpımını hesapladığını kontrol eder. Kaynak, 1'den 10'a kadar olan çarpım tablosu oluşturmayı hedefler. 'for (int i = 1; i <= 10; i++)'
Bu döngü 10 kez çalışacaktır (1, 2, 3, ..., 10 sayıları için).



Çarpım Tablosu: İç Döngü (j)

İç döngü, dış döngüdeki her bir 'i' değeri için, çarpımın tekrarlanan kısmını yönetir.

```
'for (int j = 1; j <= 10; j++)'
```

İç döngü de 10 kez çalışır.

Toplam İterasyon Sayısı: Dış döngü (10) * İç döngü (10) = 100 iterasyon.

İç İçe Döngü Akışı Görsel Analiz

Dış döngü ' $i=1$ ' olduğunda, iç döngü ' $j=1$ 'den ' $j=10$ 'a kadar 10 kez tamamlanır.
Ardından Dış döngü ' $i=2$ ' olur.
Ve iç döngü tekrar ' $j=1$ 'den ' $j=10$ 'a kadar 10 kez tamamlanır.
Bu süreç, Dış döngü sonlanana kadar ($i=10$) devam eder.



Çarpım Tablosu: String Formatlama

Her iç döngü iterasyonunda, sonuç bir metin satırı olarak formatlanır.

Bu, ListBox'a eklenecek çıktının kullanıcı dostu olmasını sağlar (Örn: '3 x 5 = 15').

Parantezler, çarpma işleminin toplama/string birleştirmeden önce yapılmasını garantiler.



Çarpım Tablosu: Ayraç Kullanımı

Her bir çarpım grubunu (örneğin 3'ler grubunu) görsel olarak ayırmak için, dış döngünün sonunda bir ayraç eklenir.

Ayraç ekleme işlemi, iç döngünün dışında, ancak dış döngünün içinde yapılmalıdır

Bu, 1'ler tablosu bittikten sonra bir çizgi, 2'ler tablosu bittikten sonra bir çizgi eklenmesini sağlar.



Proje 3: Akademik Bağlamda Zaman Karmaşıklığı

İç içe döngüler kullanıldığında ($N \times N$), zaman karmaşıklığı karesel (Quadratic) olur ve $O(N^2)$ olarak ifade edilir.

Eğer N büyükse, $O(N^2)$ algoritmalar yavaşlayabilir. 10×10 için 100 işlem, ancak 1000×1000 için 1 milyon işlem anlamına gelir.

Çarpım tablosu basit bir örnek olsa da, matris çarpımı veya büyük veri karşılaştırmaları gibi karmaşık problemlerde $O(N^2)$ analizi hayati önem taşır.

Alternatif Döngü Kontrol İfadeleri: 'break'

'break' anahtar kelimesi, döngü koşulunun doğru olup olmadığına bakılmaksızın, bulunduğu döngüyü anında sonlandırır.

Kullanım Alanları: Bir arama işleminde hedef bulunduğunda (performans için döngüyü kesmek) veya hata oluştuğunda.

İç içe döngülerde, 'break' sadece içinde bulunduğu en yakın iç döngüyü kırar, dış döngü çalışmaya devam eder.

Alternatif Döngü Kontrol İfadeleri: 'continue'

'continue' anahtar kelimesi, döngü gövdesinin geri kalanını atlar ve bir sonraki iterasyona geçer.

Kullanım Alanları: Belirli bir koşul sağlandığında o iterasyonda işlem yapmak istenmediğinde (örneğin, listedeki negatif sayıları atlamak).

'continue' çalıştırıldığında, 'for' döngüsünde doğrudan iterasyon (artırma/azaltma) ifadesine geçilir.

Döngülerde İndeksleme Stratejileri

Programlamada koleksiyonlar genellikle 0'dan başlar (Sıfır Tabanlı İndeksleme).

'for (int i = 0; i < limit; i++)' yapısı bu nedenle standarttır.

Ancak, Faktöriyel veya 1'den 100'e listeleme gibi matematiksel işlemlerde, döngüyü 1'den başlatmak ('int i = 1') daha mantıklıdır.

Üniversite öğrencileri, döngü sınırlarını (küçük mü, küçük eşit mi) belirlerken hedefin 0 tabanlı mı yoksa 1 tabanlı mı olduğunu ayırt etmelidir.



Tekrar: ListBox Metotları ve Kullanım Amaçları

`Items.Clear()`: Bir döngü başlamadan önce arayüzün boşaltılmasını sağlar. Bu, UI'nin her defasında yenilenen veriyi göstermesi için önemlidir.

`Items.Add()`: Her döngü adımının sonucunu kullanıcıya sunmanın standart yoludur. Sadece sayılar değil, formatlanmış metinler (Proje 3'teki gibi) de eklenebilir.



'for' vs. 'while' Döngüsü Karşılaştırması

'for' Döngüsü: İterasyon sayısı kesinkes bilindiğinde, başlangıç, koşul ve adım tek bir yerde tanımlanmak istendiğinde tercih edilir.

'while' Döngüsü: İterasyon sayısı belirsiz olduğunda, döngünün ne zaman biteceği bir dış olaya veya kullanıcı girişine bağlı olduğunda kullanılır.

Temel fark, 'for' döngüsünün kontrol değişkenini zorunlu olarak yapısının içine almasıdır.



Kodlama Standartları ve 'for' Döngüsü

Tek değişken kullanımı: Döngü değişkeni olarak genellikle 'i', iç içe döngülerde 'j', 'k' kullanılır (gelenekselleşmiş isimlendirme).

Anlaşılır Koşullar: Koşul ifadesi (limit) karmaşık mantıksal işlemler içermemelidir.

Kısa Gövde: Döngü gövdesi (süslü parantezler arası) mümkün olduğunca kısa ve tek bir sorumluluğa odaklanmış olmalıdır. Karmaşık işlemler metotlara ayrılmalıdır.



İleri Konu: 'for' Döngülerinde Jumps ve 'goto'

C#'ta 'goto' anahtar kelimesi, program akışını bir etikete (label) atlatmak için kullanılabilir.

Bu, teknik olarak karmaşık iç içe döngülerden çıkmak için kullanılabilir (Multi-level break).

Ancak, 'goto' kullanımı modern yazılım mühendisliği prensiplerine aykırıdır, kodu okunmaz hale getirir (Spaghetti Code) ve akademik çevrelerde önerilmez.

'break' ve 'continue' tercih edilmelidir.



Matris İşlemleri ve ' $O(N^2)$ ' Uygulaması

İç içe 'for' döngülerinin en yaygın akademik uygulaması matris (çok boyutlu dizi) işlemleridir.

Matris Tarama: Bir matrisin tüm elemanlarını ziyaret etmek için, dış döngü satırları, iç döngü sütunları temsil eder.

Matris Çarpımı: Üç iç içe döngü (' $O(N^3)$ ') gerektiren, daha karmaşık bir süreçtir ve lineer cebir uygulamalarında temeldir.

Dinamik Limitler ve Kullanıcı Etkileşimi

Proje 2'de görüldüğü gibi, döngü limitini kodda sabit ('100' gibi) tutmak yerine, kullanıcıdan alınan girdiye ('sayı') göre dinamik olarak belirlemek esastır.

Bu, programın esnekliğini ve kullanışlılığını artırır.

Giriş doğrulama (Validation): Kullanıcı girişlerinin sayı olup olmadığını ve geçerli bir aralıkta bulunup bulunmadığını kontrol etmek, sağlam kod yazımının parçasıdır.



ListBox'tan Veri Okuma

Bu hafta sadece ListBox'a veri eklemeyi öğrensek de, ListBox'ın temel amacı veri seçimi sunmaktır.

Seçilen öğeyi almak için 'listBox1.SelectedItem' veya 'listBox1.SelectedIndex' özellikleri kullanılır.

Bu özellikler, döngüler aracılığıyla yapılan işlemlere kullanıcıdan gelen geribildirimini dahil etmeyi sağlar.



Alternatif Çarpım Tablosu Gösterimleri

Proje 3'te her çarpımı ayrı satıra yazarken, iç içe döngüler kullanılarak daha kompakt, tablo formatında çıktılar da elde edilebilir.

Örneğin, iç döngü çıktıyı yan yana string olarak birleştirir, dış döngü ise satır sonunda yeni satır karakteri ekler.

Bu, RichTextBox gibi metin bazlı kontrollerde daha kullanışlı olabilir.



Döngü Optimizasyonunda 'Unrolling'

Döngü Açma (Loop Unrolling), döngü overhead'ini (yönetim maliyetini) azaltmak için iterasyonları çoğaltma tekniğidir.

Örn: 'i++' yerine 'i += 4' kullanıp döngü gövdesini 4 kez kopyalamak.

Bu, üniversite düzeyinde özellikle düşük seviye programlama veya yüksek performans gerektiren (oyun motorları) durumlarda ele alınan bir optimizasyon tekniğidir.



Döngülerde Hata Ayıklama (Debugging)

Döngü hatalarını (yanlış limit, sonsuz döngü) bulmanın en iyi yolu hata ayıklayıcı (Debugger) kullanmaktır.

Breakpoint: Kodun belirli bir noktasında durmasını sağlar.

Step Into/Over: Kodu satır satır ilerleterek her iterasyonda 'i' ve 'sonuc' gibi değişkenlerin değerlerinin nasıl değiştiğini gözlemlemeyi sağlar.

Bu, özellikle iç içe döngülerin karmaşık akışını anlamak için kritiktir.

Gelecek Haftanın Konusu: 'while' ve 'do-while'

Bu hafta 'for' döngüsü ile iterasyon sayısı bilinen görevlere odaklandık. Gelecek hafta, koşula dayalı döngüler olan 'while' ve 'do-while' yapılarını inceleyeceğiz.

Bu döngüler, dosya okuma/yazma, kullanıcı girişi beklerken veya ağ bağlantısı devam ederken gibi, iterasyon sayısının önceden bilinmediği senaryolarda kullanılır.



Özet ve Temel Çıkarımlar

'for' döngüsü, belirli sayıda tekrarlı işlemi kolayca yönetir.

Başlangıç (i), Koşul ($\leq$ limit) ve Adım (i++) tanımları zorunludur.

ListBox, döngü ile üretilen verileri son kullanıcıya temiz bir şekilde sunmak için idealdir.

Faktöriyel gibi hesaplamalarda veri tipi (int yerine long) seçimi, taşmayı önlemek için hayati önem taşır.

İç içe döngüler ($O(N^2)$) karmaşık tablolar (Çarpım Tablosu) ve matris işlemleri için kullanılır.

Soyut Düşünce: Döngüler ve Matematiksel Seriler

'for' döngüsü, matematiksel serilerin (örneğin Fibonacci serisi, geometrik seriler) hesaplanması için temel araçtır.

Faktöriyel hesaplamasına benzer şekilde, toplama ('+') veya çarpma ('*') operatörleri kullanılarak serilerin N. elemanı veya ilk N elemanının toplamı bulunabilir.



Faktöriyel Örnek: Taşma Sınırı Deneyi

Eğer Proje 2'de 'long' yerine 'int' kullanılsaydı:

'int sonuc = 1;'

Genellikle 13! hesaplanmaya çalışıldığında, sonuç pozitif bir sayı yerine çok büyük bir negatif sayı olarak görünürdü (Taşma sonrası en küçük negatif değere sarması - wrapping).

Bu, kritik sistemlerde veri kaybına ve hatalı kararlara yol açar.

Koleksiyonlarda Performans: 'for' vs 'foreach' (Enrichment)

'for' döngüsü, 'List<T>' ve Array'ler gibi indekslenebilir koleksiyonlarda direkt bellek adresine erişim sağladığı için genellikle 'foreach'e göre çok az daha hızlıdır.

Ancak modern C# derleyicileri ve JIT (Just-In-Time) optimizasyonları bu farkı minimize eder.

Temel kural: İndekse ihtiyacınız varsa 'for', sadece elemanları okumak istiyorsanız 'foreach' kullanın.

İç İçe Döngü Alternatifi: Recursive Fonksiyonlar

Faktöriyel hesaplaması, 'for' döngüsü (iteratif çözüm) yerine özyinelemeli (recursive) bir fonksiyonla da yazılabilir.

Rekürsif çözüm, kodun daha temiz ve matematiksel olarak daha doğal görünmesini sağlar.

Ancak büyük N değerleri için yığın taşması (Stack Overflow) riski ve performans düşüşü yaratabilir. Üniversite öğrencileri her iki çözümü de bilmelidir.



Kullanıcı Deneyimi (UX) ve Döngü Çıktıları

Eğer bir döngü çok uzun sürüyorsa (örneğin 100 milyondan fazla iterasyon), UI donar (Freezing UI). Bu kötü bir kullanıcı deneyimidir.

Çözüm: Uzun süren döngüleri arka planda çalıştırmak (Asenkron Programlama - 'async' / 'await' veya Task'lar kullanarak).

Bu ileri konu, döngülerin gerçek dünya uygulamalarındaki etkisini anlamak için önemlidir.



Döngü Kontrolü: Tekrarlayan Kullanıcı Girişleri

Eğitimde gösterilen 'TextBox', kullanıcı girişini alırken, döngüler girişin geçerli olmasını sağlamak için kullanılabilir.

Örn: 'do-while' döngüsü, kullanıcı geçerli bir sayı girene kadar (try-catch kullanarak) tekrar tekrar sormak için idealdir. (Bu, gelecek haftanın konusuna geçiş sağlar).



ListBox'ta Sıralama ve Filtreleme

Döngü ile doldurulan ListBox verileri, kullanıcıya sunulmadan önce sıralanabilir veya filtrelenebilir.

ListBox'ın kendi 'Sorted' özelliği olsa da, profesyonel uygulamalarda listeyi doldurmadan önce verilerin (örneğin LINQ kullanarak) sıralanıp string olarak eklenmesi daha yaygındır.



Görsel Programlamada Label Kontrolü

Label kontrolü, Faktöriyel örneğinde olduğu gibi, tek ve kesin sonuçları (döngü çıktılarını) göstermek için kullanılır.

Genellikle 'TextBox'tan farklı olarak, 'Label'a programatik olarak yazı yazılır ve kullanıcının doğrudan değiştirmesi engellenir. Bu, sonucu garanti altına alır.



Proje 1'in Adaptasyonu: Sadece Çift Sayıları Ekleme

1'den 100'e sadece çift sayıları eklemek için, 'for' döngüsünün içine bir 'if' koşulu eklenebilir:

```
'for (int i = 1; i <= 100; i++)'
```

```
'{ if (i % 2 == 0) { listBox1.Items.Add(i); } }'
```

Bu, sadece iterasyon adımı 2 artırılmadığında kullanılan alternatif bir yaklaşımdır.

Proje 2: Negatif Sayılar ve Hata Yönetimi

Faktöriyel sadece pozitif tam sayılar için tanımlıdır (ve sıfır için $0!=1$). Programın, 'TextBox"a negatif bir sayı girildiğinde doğru şekilde tepki vermesi gerekir.

Bu, 'if (sayi < 0)' kontrolü ile sağlanır ve kullanıcıya bir hata mesajı gösterilmelidir (MessageBox veya Label üzerinde).

İç İçe Döngülerde Optimizasyon

İç döngüyü mümkün olduğunca basit tutun.

Dış döngü içinde hesaplanabilecek sabit değerleri iç döngünün dışına taşıyın.

Örn: Bir hesaplama sonucu her iterasyonda aynı kalacaksa, bunu dış döngünün bir adımında hesaplayın.



Veri Yapıları ve Döngü Seçimi

Diziler (Array) ve 'List<T>' gibi indekslenebilir yapılar için 'for' döngüsü doğal bir seçimdir.

Hash tabloları ('Dictionary<T>') gibi sırasız koleksiyonlar üzerinde indeks tabanlı gezinme mantıksızdır; bu durumda 'foreach' kullanılır.

Programcı, kullandığı veri yapısına göre en uygun döngü yapısını seçmelidir.



Döngü İfadelerinde Tür Dönüşümü

Döngü değişkenleri (örneğin 'int i'), UI kontrollerine ('ListBox.Items.Add()') eklenirken veya stringlerle birleştirilirken otomatik olarak metne dönüştürülür (Implicit Conversion).

Ancak, 'TextBox'tan veri alırken, metni tamsayıya dönüştürmek gerekir ('Convert.ToInt32' veya 'int.Parse') (Explicit Conversion).



C# Döngü İfadelerinde 'var' Kullanımı

C# 3.0 sonrası 'var' anahtar kelimesi ile yerel değişkenin tipi derleyici tarafından çıkarılabilir.

```
'for (var i = 0; i < limit; i++)'
```

Bu, koda esneklik katsa da, kaynakta 'int i' kullanıldığı için (açık tip belirtme) bu daha geleneksel ve önerilen yaklaşımdır.

Açık tip tanımlama, büyük kod tabanlarında okunabilirliği artırır.

Sonuç: Döngülerin Temel Mimari Rolü

Döngüler, programlamada otomasyonun ve büyük veri işleme kapasitesinin temelini oluşturur.

'for' döngüsü, özellikle tekrarlama sayısının net olduğu, algoritmanın deterministik olduğu senaryolar için en sağlam ve okunabilir yapıdır.

Programcıların temel görevlerinden biri, bir problemi döngü mantığına dönüştürmektir (iteratif düşünce).

Ek Uygulama: Asal Sayı Bulma

Asal sayı bulma algoritması, bir sayının 1'den kendisine kadar olan sayılara tam bölünüp bölünmediğini kontrol etmek için bir döngü kullanır.

Belirli bir aralıktaki tüm asal sayıları bulmak için ise iç içe döngü yapısı kurulmalıdır (Dış döngü aralık, iç döngü bölme kontrolü).



C# GUI Programlamada Olay Yönetimi

Döngü kodlarının tamamı, genellikle bir olaya (event) tepki olarak çalışır. Projelerde bu olay, 'Button Click' olayıdır.

Olay Yönetimi: Kullanıcı etkileşiminin (mouse tıklaması) bir metodu tetiklemesini sağlayan mekanizmadır. Bu, programın kullanıcı odaklı ve tepkisel çalışmasını sağlar.



Döngü Değişkeni İsimlendirme Kuralları

Tekrarlama amaçlı kullanılan döngü değişkenleri (i, j, k), programlamada kısa tutulabilir.

Ancak, döngünün amacı bir koleksiyondaki indeksi temsil ediyorsa, daha açıklayıcı isimler ('rowIndex', 'colIndex') kullanmak daha iyi bir akademik uygulamadır.

Üniversite düzeyinde projelerde, değişken isimlerinin amacını yansıtması beklenir.



Döngü ve Koşul İfadelerinin Güvenliği

Koşul ifadesinde yanlış operatör kullanımı (örneğin ' $i < \text{limit}$ ' yerine yanlışlıkla ' $i \leq \text{limit}$ ' yazmak), 'Off-by-One Error' olarak bilinen yaygın bir hataya neden olur.

Bu tür hatalar, dizinin sınırlarını aşmaya (`IndexOutOfRangeException`) ve programın çökmesine yol açabilir.



C# ListBox Gelişmiş Özellikler

ListBox, birden fazla öğe seçimi yapılmasına izin verebilir (MultiSelect). Bu, verilerin toplu işlenmesi gerektiğinde önemlidir.

SelectionMode özelliği, tek seçim (One), çoklu seçim (MultiSimple) veya genişletilmiş çoklu seçim (MultiExtended) olarak ayarlanabilir.



For Döngüsünü Fonksiyonel Hale Getirmek

Modern C# (LINQ), döngüleri daha deklaratif bir şekilde yazma imkanı sunar.

Örn: 1'den 100'e sayı üretmek için:

`'Enumerable.Range(1, 100).ToList()'`

Bu yapı, özellikle ListBox'a veri eklemekten önce veriyi hazırlarken döngü yerine tercih edilebilir.

Döngülerin Kullanıldığı Gerçek Dünya Senaryoları

Finans: Aylık faiz hesaplamaları veya taksit planları oluşturma.

Oyun Geliştirme: Oyun haritasındaki tüm nesneleri veya düşmanları güncelleme.

Veri Analizi: Bir log dosyasındaki binlerce satırı okuma ve işleme.

Tüm bu senaryolar, temel 'for' döngüsü yapısına dayanır.



Özet: Proje Amaçlarının Tekrarı

Proje 1: Temel 'for' yapısı ve 'ListBox.Items.Add()' kullanımı.

Proje 2: Hesaplama (Accumulator) deseni ve 'long' veri tipinin gerekliliği.

Proje 3: İç içe döngülerin (Nested Loops) çok boyutlu çıktı oluşturmadaki rolü.

Bu projeler, C# programlamada iterasyonun temelini sağlamlaştırmaktadır.



Tekrarlı İşlemler ve Kontrol Akışı

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

